

AI Hardware Compiler: A VLIW Approach to Instruction Scheduling for Accelerator Architectures

Authors

Heng-I Chu

Sahil Patel

Mikhail Golovenchits

chu244@purdue.edu

pate1701@purdue.edu

mgoloven@purdue.edu

Senior Design

Senior Design

Senior Design

Acknowledgments

Vihaan Chinthakindi

Volunteer

Elmore School of Electrical And Computer Engineering

Purdue University, West Lafayette, IN

Submitted To:

Fan Jing Hoon

Malcolm McClymont

Sooraj Chetput

Timothy Hein

Rishikesh Bhatina

Abstract

The Atalla AI Accelerator coprocessor requires a complex programming model, allowing for direct control of the processor system. Using a compiler, we can alleviate hardware complexity and boost processor performance. To support the Atalla ISA, we modify the C frontend and create a custom backend architecture. We implemented this using PPCI (Purely Python Compiler Infrastructure), an easily extendable research-focused open-source compiler project. To the frontend, we added new bfloat16 and vector datatypes to accommodate vector register functionality of the processor, as well as intrinsic functions such as GEMM to support matrix multiplication. In the backend, we added a new and custom architecture, built by extending RISC-V to support the Atalla ISA for both scalar and vector operations. Furthermore, we propose an additional compiler pass after assembly generation to group instructions into VLIW packets for explicit instruction-level parallelism. These packets are constructed with explicit latency modeling and hardware dependency tracking, essentially moving the burden of instruction scheduling onto the software, reducing hardware complexity and cost. Overall, we produced a functioning compiler conforming to ISA requirements. This compiler is able to produce the entire scalar part of the ISA, including floating-point variables. The compiler is also capable of correct and efficient VLIW packetization on these scalar instructions. In order to provide complete support to the Atalla ISA, we need to finalize ISA patterns for vector instructions, add SDMA and other intrinsic functions similar to GEMM, modify register allocation with hardware register banking requirements, and finally formally verify correctness and performance on an upcoming simulator. This semester, we delivered a proof-of-concept compiler with core scalar instruction functionality. Next semester, we aim to finish the remaining work to fully support the Atalla ISA and begin exploring compiler applications.

Table of Contents

1. Introduction	4
1.1 Problem Statement	4
1.2 Significance	4
1.3 Prior Designs	5
1.4 Background	5
1.4.1 What is a Compiler?	5
1.4.2 What is an ISA?	7
1.4.3 What is VLIW Packetization?	8
2. Design Choices	8
2.1 Compiler Infrastructure	8
2.2 Source Language	9
2.3 Vector Operations	9
2.4 Packetization Approach	11
3. Solution	12
3.1 Frontend	12
3.2 Backend Changes	16
3.2.1 Atalla Architecture	16
3.3 VLIW Packetization	19
3.3.1 Scoreboarding	19
3.3.2 Packetization	21
4. Results	22
4.1 Scalar Outputs	22
4.2 Packetization Example Cases	24
5. Discussion of Results	26
6. Limitations	26
7. Conclusion	27
8. Citations	28

1. Introduction

1.1 Problem Statement

The Atalla AI Accelerator Coprocessor system requires a compiler to translate from C code into the target hardware-specific ISA. The programmer should have the ability to create, load data into, and perform operations like matrix multiplication on vector registers, which is not a feature of classical C. Furthermore, as the vectors contain brain floating-point data, the programmer should be able to declare variables of this type and use them to populate vectors. This would require changes to the frontend phase of the compiler. The compiler must issue code with full adherence to the Atalla ISA, which has been provided to us. Additionally, to maximize the efficiency of instruction scheduling, the compiler must issue assembly in 4 instruction-wide VLIW packets, organized by hardware dependencies, instruction latencies, and control-flow organization. Since the packetization would be completed at compile-time, this allows a reduction in the complexity of the hardware, which would otherwise be responsible for this grouping, driving up the hardware costs dramatically.

1.2 Significance

The problems we are solving matter because they directly affect the system's performance, efficiency, and correctness. Without automatic instruction packetization and scheduling, developers would have to manually handle hardware constraints such as dependency tracking, register allocation, read-write hazards, and other low-level details that are both time-consuming and easy to get wrong. In addition, developers would struggle to fully utilize the processor's parallel functional units and VLIW datapath, resulting in reduced throughput and inefficient execution. Therefore, a systematic solution of these challenges not only enables the processor to reach its intended performance but also preserves correctness by consistently enforcing the architectural rules.

1.3 Prior Designs

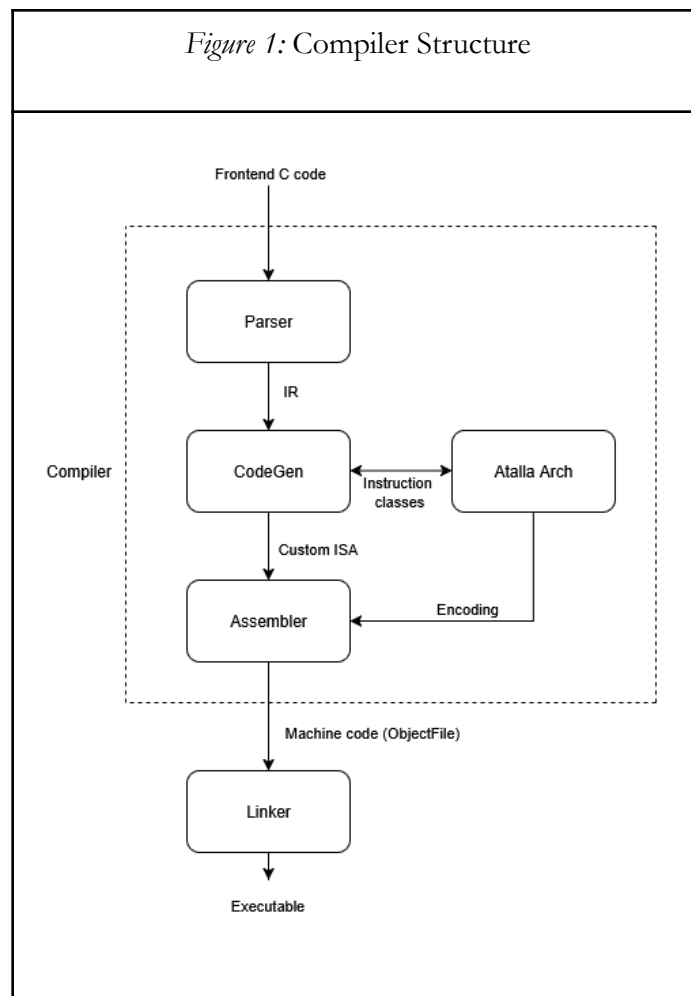
In prior designs, our team has attempted to create a compiler for our custom tensor processor and Instruction Set Architecture (ISA) using LLVM, a C++ based production compiler infrastructure. An introduction of LLVM can be found in this research paper [1]. However, this approach proved to be difficult due to the extensiveness of the LLVM code base and the complexity of adding hardware-specific optimization techniques. Furthermore, as hardware continued to evolve, new features such as multiple specialized functional units, variable latency components, register banking, and a more complex scoreboard model have introduced more challenging constraints that LLVM was not designed to express cleanly. More importantly, the processor has adopted a VLIW execution model in which the hardware expects instructions to be emitted in packets of four for better parallelization, which LLVM also does not support.

1.4 Background

1.4.1 What is a Compiler?

A compiler is a software system that converts from one programming language (the source) to another (the target). In our case, the target language is the custom assembly used by the AI Hardware team to represent the Atalla-specific ISA. The general purpose of a compiler is to translate from human-readable code into machine-readable code, since machines cannot understand instructions directly and instead need to be instructed using binary code. To do so, a compiler is split into two parts, the frontend and backend, each consisting of several functional components. In the frontend, the parser reads source code and uses lexical and syntax analysis techniques to generate an Abstract Syntax Tree (AST), which contains the syntactic structure of a program [2]. After the tree is generated, the compiler traverses it to verify the correctness of the syntax, then generates the

Intermediate Representation (IR). IR is a language that is universal for the compiler and independent of both the source and the target, in order to allow the compiler to be configurable for multiple sources and targets and maintain logic independently of the two [3]. The optionally optimized IR is then fed into the backend section of the compiler, specifically the Code Generation phase (CodeGen for short). During this phase, the compiler again traverses the AST, now containing IR, and matches the IR to ISA patterns specified in the target architecture to generate machine code. After the machine code is generated, we can also optionally optimize the code in accordance with the specifications of the machine it will be run on, this is called machine-specific optimization.



1.4.2 What is an ISA?

The ISA (Instruction Set Architecture) defines the instructions, data types, registers, and the programming interface for managing main memory. It acts as an interface between the hardware and software. The hardware is designed around the ISA to implement its instructions and architecture. There are general-purpose ISAs such as RISC-V and x86 that allow for a wide range of computing tasks. We are using a domain-specific ISA in order to shape the architecture around the specific use cases of the hardware, which are computing matrix multiplication and running AI kernels. In Atalla's architecture, there are scalar, vector, GEMM (General Matrix Multiply), and SDMA (Scratchpad Direct Memory Access) operations, all serving the purpose of matrix multiplication – the core operation behind machine learning. The Atalla ISA bitspec can be found in Figure 2.

Figure 2: Atalla ISA Bit-Spec

	39	38	37	36	35	34	33	32	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0																				
R											rs2							rs1							rd							opcode - 7 bits																												
BR	imm9										rs2							rs1							incr-imm7							opcode - 7 bits																												
I											imm12							rs1							rd							opcode - 7 bits																												
M											imm12							rs1							rd							opcode - 7 bits																												
MI	imm25																								rd							opcode - 7 bits																												
S																																								opcode - 7 bits																				
VV						sac		mask			vs2							vs1							vd							opcode - 7 bits																												
VS											mask			rs1							vs1							vd							opcode - 7 bits																									
VI	imm5					mask			imm8							vs1							vd							opcode - 7 bits																														
VM	rc_id					rc		sid		num_rows					num_cols					rs1							vd							opcode - 7 bits																										
SDMA											sid		num_rows					num_cols					rs2							rs1 and rd1							opcode - 7 bits																							
MTS																																								vms							rd							opcode - 7 bits						
STM																			rs1							vmd							opcode - 7 bits																											

1.4.3 What is VLIW Packetization?

Very Long Instruction Word (VLIW) is an architectural design that allows the machine to issue multiple independent instructions in a single large piece. Instead of implementing an out-of-order or multithreading scheduler in hardware to achieve instruction-level parallelism, VLIW moves this responsibility to the compiler and software. This not only makes the hardware simpler, but also allows the compiler to schedule parallel operations in the same way as it would with far less hardware control logic. Packetization is the process that combines lines of independent assembly instructions into grouped wide instructions that will be fed into the machine to execute the instructions simultaneously. However, more than just simply stacking instructions, the compiler would have to analyze dependencies between operations, determine which instructions can be safely executed side by side without any conflicts or hazards, and group them into fixed-width packets that match the hardware constraints. Packetization ensures independent instructions can be executed in parallel while preserving the various dependencies.

Section 2: Design Choices

2.1 Compiler Infrastructure

We considered and researched several compiler infrastructures, the main two being PPCI and LLVM, in addition to the idea of building a custom compiler from scratch. At first, we were mainly interested in LLVM for its production-level software and vast library. LLVM's core libraries are written in C++, making the compiler infrastructure more efficient and better optimized [1]. We also considered PPCI (Pure Python Infrastructure Library), which is written entirely in Python. The documentation for PPCI can be found here [5]. This library was easier to understand and had a smaller code base compared to LLVM, though it is a pre-production software that contains

unfinished execution paths and features. We decided not to build a custom compiler from scratch due to the time constraints of researching and creating a compiler. We also chose not to use LLVM because of its highly vast library that would also present a time constraint with its lower-level language code base that is, by nature, farther away from natural language and is therefore harder for programmers to read. PPCI was chosen for its simpler process of reading code and understanding libraries, adding and extending architectures, optimizations, and implementing new compiler passes.

2.2 Source Language

The ideal source language we would have liked to use was C++, since the programming model we were provided was based on that language. However, PPCI currently does not support C++ as a source [5]. We had the option of creating a custom new parser to be able to process C++, but ultimately decided against it due to the time constraints. Instead, we elected to use one of the existing parsers within PPCI. The supported languages are Basic, C, Fortran, LLVM IR, OCaml, Pascal, and Python. Out of these options, we chose C as the source language to base our implementation on. We made this decision due to C's popularity, similarity to C++, as well as its closeness to hardware, since the programmer does need to be aware of specific datatypes and memory operations the system will perform, much akin to embedded software programming. This requirement also filters out higher-level languages like Python, where datatypes are often implicit, and the programming structure is significantly more abstracted from the hardware.

2.3 Vector Operations

In order to create and manipulate vector registers, we elected to create a “vector” datatype for the programmer to load values into and perform matrix multiplication on. Initially, we had considered simply maintaining an array of bf16s to represent the vectors; however, this would

significantly complicate GEMM operations due to the pointer logic involved with arrays in C, whereas if we mapped vector registers to vector-type variables, the loading of data into these registers can be handled by the vector-load function. Since we still need to load bfloat16 data into the vectors, we had to implement the bfloat16 datatype into the compiler, so that the user can create an array of up to 32 bf16 values, and load them into the vectors using a vector-load function. Bfloat16, or bf16 for short, stands for Brain Floating Point of 16 bits. This datatype maintains the same range as a 32-bit float, at the expense of mantissa precision bits [4]. We could not simply reuse the existing PPCI float implementation because PPCI uses 32-bit and 64-bit floats. Furthermore, in the Atalla ISA, only 16-bit floats are supported. In order to do this, we had the option of introducing a new datatype for bfloat16 or reconfiguring existing float definitions to consist of 16 bits. Both of these solutions were equally valid, but we decided to pursue changing the float definition to 16 bits, since the ISA does not allow 32 and 64-bit floats, as mentioned above. This would have required us to change the standard C parser, so we decided to make a copy of the C parser in order not to break existing C logic.

Next, there was the question of implementing GEMM operations in the frontend. The first approach we considered was to use inline assembly to call the GEMM instruction directly. If configured correctly, this would require the least work on our part, since we simply needed to call “gemm vr, v1, v2” to perform this operation. There were 2 issues with this approach. The first issue is that when calling inline assembly, the compiler is directed to emit the instruction to the output stream without optimization, which could violate the scheduling requirement. The other issue amounted to PPCI’s pre-production status, so inline assembly functionality was not yet fully implemented. This meant that we would need to allocate resources and time to configure this major feature of the compiler. Eventually, we landed on the idea to create a new intrinsic function for the GEMM operation. Intrinsic functions are functions like `sizeof()`, which the compiler lowers to their

associated use, without the need to be defined by the user. This would significantly lighten the load on the programmer, as they would simply need to call the function, without specifying the exact registers, instead simply using the vector variables we defined above. Implementing new intrinsic functions will require modifying several different parts of the compiler, so this solution is a lot more work-intensive. The advantages of this solution significantly outweighed the negatives, so we decided to pursue implementing GEMM as an intrinsic function within AtallaC.

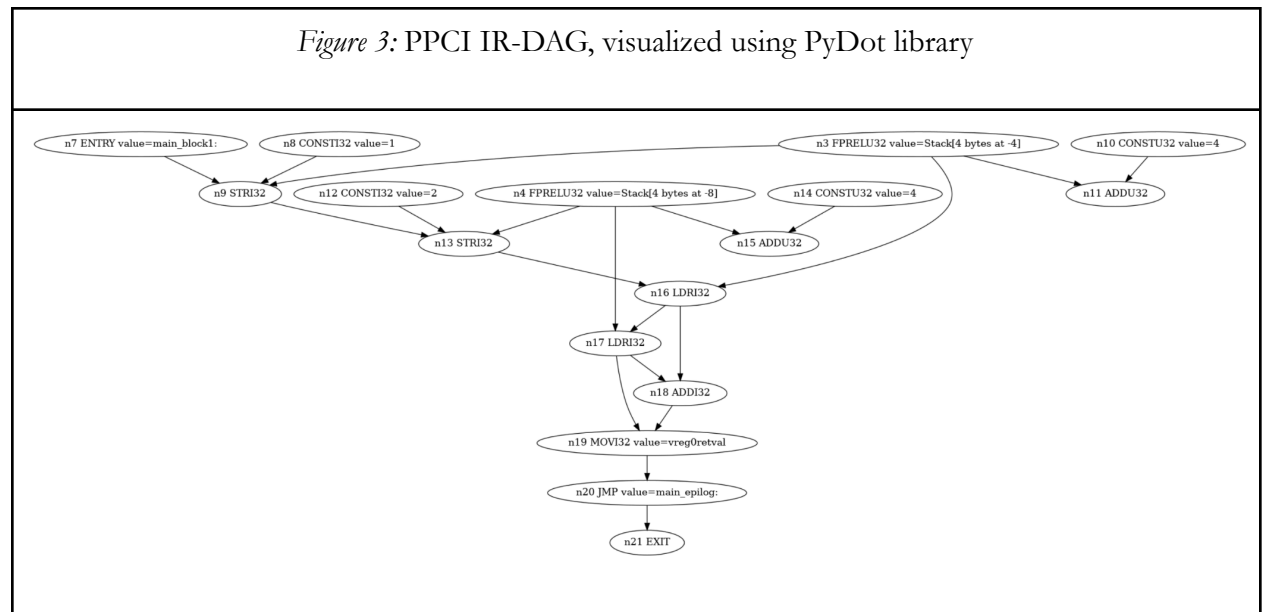
2.4 Packetization Approach

During the investigation stages of the packetizer development, we have consulted several experts in the field to explore different points where the VLIW packetizer can be implemented. We learned that the packetizer could be placed in one of two stages: after IR generation, or after assembly code emission. The first option offers a more rigid abstraction of the source code while still leaving us space for custom assembly code generation. This can be done by reordering and reselecting the IR according to our hardware constraints. The second option offers the most straightforward approach, yet it will be restricted by the compiler's existing code-generation algorithm and leaves us little room for additional optimization. This can be done by adding another pass of the generated assembly, which directly selects assembly instructions into packets that meet our constraints.

At first, we tried to place the VLIW packetizer directly after the IR generation. The idea was that keeping everything at a higher level would preserve structural information while still giving us enough freedom for optimization. In addition, we also noticed that PPCI already provides an Intermediate Representation Directed Acyclic Graph (IR-DAG), which seemed convenient for modeling dependencies. This IR-DAG can be found in Figure 3. However, we later learned that the IR-DAG still operates on virtual registers, unresolved instruction selection, and without

machine-level scheduling. Because of this, trying to group IR nodes will either produce illegal packets or packets that cannot be validated once lowered. The more critical problem was that packetization fundamentally depends on the final machine code and the true hardware dependencies, which only exist after assembly generation and register allocation. Given these limitations, we abandoned the IR level approach and decided to build the packetizer after assembly code generation, where instructions are fully lowered, and basic blocks are concretely defined.

Figure 3: PPCI IR-DAG, visualized using PyDot library



Section 3: Solution

3.1 Frontend

The copied AtallaC parser already contained support for the necessary integer datatypes from its C parser base, so nothing else needed to be added to support integers. In order to allow the parser to correctly generate vector and bf16-type AST nodes, we first needed to define the IR types the parser would be able to emit. As can be observed in Code Snippet 1, we added a definition for

bf16 as a FloatingPointType consisting of 16 bits, and vec as a VectorType consisting of 32*16 bits, since vectors store 32 bf16 values.

Code Snippet 1: New & Existing IR Types (ppci/ir.py)

```
# The builtin types:
f64 = FloatingPointTyp("f64", 64) #: 64-bit floating point type
f32 = FloatingPointTyp("f32", 32) #: 32-bit floating point type
i64 = SignedIntegerTyp("i64", 64) #: Signed 64-bit type
i32 = SignedIntegerTyp("i32", 32) #: Signed 32-bit type
i16 = SignedIntegerTyp("i16", 16) #: Signed 16-bit type
i8 = SignedIntegerTyp("i8", 8) #: Signed 8-bit type
u64 = UnsignedIntegerTyp("u64", 64) #: Unsigned 64-bit type
u32 = UnsignedIntegerTyp("u32", 32) #: Unsigned 32-bit type
u16 = UnsignedIntegerTyp("u16", 16) #: Unsigned 16-bit type
u8 = UnsignedIntegerTyp("u8", 8) #: Unsigned 8-bit type
ptr = PointerTyp("ptr") #: Pointer type

# Atalla specific types
bf16 = FloatingPointTyp("bf16", 16) #: 16-bit floating point type
vec = VectorTyp("vec", 32*16) #: Vector type
```

As we established previously, we removed 32 and 64-bit floats from this parser and added the bfloat16 datatype. This will generate a bf16 IR AST Node upon reading the ‘float’ keyword, which the code generator will be able to encode into assembly accordingly. The type map, which is what defines which datatypes are available to the parser, can be seen in Code Snippet 2. Here, we also define a basic vector type to allow “vec” variables to be used. Since Python stores its floats using 32 bits, we also created functions to convert from f32 to bf16 and vice versa. These functions will be used during the code generation phase to translate between floats stored in the AST and the necessary machine code.

Code Snippet 2: Atalla C Parser's IR Type Map (ppci/lang/atalla_c/codegenerator.py)

```
self.ir_type_map = {
    BasicType.CHAR: (ir.i8, 1),
    BasicType.UCHAR: (ir.u8, 1),
    BasicType.SHORT: (ir.i16, 2),
    BasicType.USHORT: (ir.u16, 2),
    BasicType.INT: (int_types[int_size], int_size),
    BasicType.UINT: (uint_types[int_size], int_size),
    BasicType.LONG: (int_types[long_size], long_size),
    BasicType.ULONG: (uint_types[long_size], long_size),
    BasicType.LONGLONG: (ir.i64, 8),
    BasicType.ULONGLONG: (ir.u64, 8),

    BasicType.FLOAT: (ir.bf16, 2), # New 16-bit Float

    ## Float types removed from C standard
    # BasicType.FLOAT: (ir.f32, 4),
    # BasicType.DOUBLE: (ir.f64, 8),
    # BasicType.LONGDOUBLE: (ir.f64, 8),

    BasicType.VA_LIST: (ir.ptr, self.ptr_size),
    BasicType.VECTOR: (ir.vec, 64), # New vector datatype
}
```

Next, we implemented the GEMM intrinsic function. To achieve this, we added “gemm” into the list of keywords the parser scans for. Upon reading the “gemm” keyword, the parser will consume the arguments and output the AST node created by *semantics.on_gemm()*. This logic is displayed in the figure below.

Code Snippet 3: Atalla C Parser's IR Type Map (ppci/lang/atalla_c/codegenerator.py)

```
elif self.peek == "gemm":
    location = self.consume("gemm").loc
    self.consume("(")
```

```

args = []
while self.peek != " ":
    args.append(self.parse_primary_expression())
    if self.peek != " ":
        self.consume(",")
expr = self.semantics.on_gemm(args[0], args[1], args[2], location)
self.consume(" ")
}

```

After the IR is generated, the IRDAG must generate instruction selection trees for the ISA patterns to later consume. Our implementation of the `do_gemm` IRDAG function can be seen in the code below. This function is called when CodeGen reads a GEMM AST node and outputs a GEMM selection tree. This concludes the changes we made to the frontend; however, we still had to modify the backend to ensure that the new operations are correctly converted to assembly.

Code Snippet 4: do_gemm IRDAG function (ppci/codegen/irdag.py)

```

def do_gemm(self, node):
    # Gemm.ir stores its operands in node.value as a list
    args = list(node.value)

    reg_outputs = []
    for arg in args:
        # get SGValue for the argument
        arg_val = self.get_value(arg)

        # allocate a new vreg for a u32 register and move the value into it
        vreg = self.new_vreg(ir.vec)
        mov_node = self.new_node("MOV", ir.vec, arg_val, value=vreg)
        self.chain(mov_node)

        # create a REG node that represents the vreg and expose its output
        reg_node = self.new_node("REG", ir.vec, value=vreg)
        out = reg_node.new_output(vreg.name)
        out.vreg = vreg
        reg_outputs.append(out)

    # Create the GEMMVV node taking the three register outputs as inputs
    sgnode = self.new_node("GEMMVEC", None, *reg_outputs)
    self.debug_db.map(node, sgnode)
    self.chain(sgnode)

```

3.2 Backend Changes

3.2.1 Atalla Architecture

PPCI uses “architectures” to contain the code generator logic for each target language, so we created a new architecture for the Atalla processor. The Atalla architecture is generally based on the RISC-V implementation in PPCI, since RISC-V was the most similar existing ISA to the Atalla ISA out of the ones that were available by default. The architectures in PPCI mainly consist of the following Python files: `instructions.py` is where all instruction behavior, fields, and opcodes are defined; `registers.py` contains the hardware registers available for use; `tokens.py` contains the token class definitions, which are used to define the bit layout of each instruction type such as its opcode, register, immediates and mask fields for the various instruction types. Atalla instructions consist of 40 bits and use different types than RISC-V, so we rewrote the tokens and instructions files from scratch. We defined all the instruction type tokens and behaviors in accordance with the Atalla ISA (refer to 1.4.2 Figure 2). An example of tokens and instruction definitions for R-type instructions can be found in code snippets 5 and 6 below.

Code Snippet 5: Atalla R Tokens (ppci/arch/atalla/tokens.py)

```
class AtallaRToken(Token):
    class Info:
        size = 40
        opcode = bit_range(0, 7)
        rd = bit_range(7, 15)
        rs1 = bit_range(15, 23)
        rs2 = bit_range(23, 31)
        reserved = bit_range(31, 40)
```



```
class AtallaRInstruction(Instruction):
    tokens = [AtallaRToken]
    isa = isa

def make_r(mnemonic, opcode):
    rd = Operand("rd", AtallaRegister, write=True)
    rn = Operand("rn", AtallaRegister, read=True)
    rm = Operand("rm", AtallaRegister, read=True)
    syntax = Syntax([mnemonic, " ", rd, ",", " ", rn, ",", " ", rm])
    tokens = [AtallaRToken]
    patterns = {
        "opcode": opcode,
        "rd": rd,
        "rs1": rn,
        "rs2": rm,
    }
    members = {
        "syntax": syntax,
        "rd": rd,
        "rn": rn,
        "rm": rm,
        "patterns": patterns,

        "tokens": tokens,
        "opcode": opcode,
    }
    name = mnemonic.title() + "R"

    return type(name, (AtallaRInstruction,), members)

# R-types:
Adds = make_r("add_s", 0b00000001)
Subs = make_r("sub_s", 0b00000010)
Muls = make_r("mul_s", 0b00000011)
Divs = make_r("div_s", 0b00000100)
Mods = make_r("mod_s", 0b00000101)
Ors = make_r("or_s", 0b00000110)
Ands = make_r("and_s", 0b00000111)
Xors = make_r("xor_s", 0b00001000)

Slls = make_r("sll_s", 0b00001001)
Srls = make_r("srl_s", 0b00001010)
Sras = make_r("sra_s", 0b00001011)
Slts = make_r("slt_s", 0b00001100)
```

```
Sltus = make_r("sltu_s", 0b0001101)
```

In addition to the scalar registers and instructions, we also defined vector registers and instructions to implement the necessary vector operations. An example of VV-type vector instructions can be found in code snippet 7. These are instructions that operate on two vector registers. GEMM is an example of one of such instructions.

Code Snippet 7: Atalla VV Instructions (ppci/arch/atalla/vector_instructions.py)

```
0):    def make_vv(mnemonic: str, opcode: int, *, default_mask: int = 0, default_sac: int =
        vd = Operand("vd", AtallaVectorRegister, write=True)
        vs1 = Operand("vs1", AtallaVectorRegister, read=True)
        vs2 = Operand("vs2", AtallaVectorRegister, read=True)
        syntax = Syntax([mnemonic, " ", vd, ",", " ", vs1, ",", " ", vs2])

        patterns = {"opcode": opcode, "vd": vd, "vs1": vs1,
                    "vs2": vs2, "mask": default_mask, "sac": default_sac}
        members = {"syntax": syntax, "vd": vd, "vs1": vs1,
                  "vs2": vs2, "patterns": patterns, "opcode": opcode}
        return type(mnemonic.replace(".", "_"), (AtallaVVInstruction,), members)

# VV
AddVv = make_vv("add_vv", 0b0101000)
SubVv = make_vv("sub_vv", 0b0101001)
MulVv = make_vv("mul_vv", 0b0101010)
DivVv = make_vv("div_vv", 0b0101011)
AndVv = make_vv("and_vv", 0b0101100)
OrVv = make_vv("or_vv", 0b0101101)
XorVv = make_vv("xor_vv", 0b0101110)
GemmVv = make_vv("gemm_vv", 0b0101111)
MgtVv = make_vv("mgt_vv", 0b0110000)
MltVv = make_vv("mlt_vv", 0b0110001)
MeqVv = make_vv("meq_vv", 0b0110010)
MneqVv = make_vv("mneq_vv", 0b0110011)
```

In order to translate code from the IR AST to assembly, PPCI uses a matching technique called ISA patterns. This is done by comparing selection trees against a set of predefined patterns, each of which is associated with certain functions that output the required assembly instructions. PPCI has existing pattern-matching functionality for RISC-V that we adapted for Atalla. An example of one of these ISA patterns for “mul.s” can be seen in the code snippet below. Here, MULU32(reg, reg) is one of the selection tree patterns, which means that we are multiplying 2 registers that contain 32-bit unsigned integers. After the patterns match the selection tree, the function emits the mul.s instruction acting on the source registers and returns the target register containing the result of the multiplication.

Code Snippet 8: ISA Patterns for Multiplication (ppci/arch/atalla/tokens.py)

```
@isa.pattern("reg", "MULI8(reg, reg)", size=10)
@isa.pattern("reg", "MULU8(reg, reg)", size=10)
@isa.pattern("reg", "MULU16(reg, reg)", size=10)
@isa.pattern("reg", "MULI32(reg, reg)", size=10)
@isa.pattern("reg", "MULU32(reg, reg)", size=10)
def pattern_mul_i32(context, tree, c0, c1):
    d = context.new_reg(AtallaRegister)
    context.emit(Muls(d, c0, c1))
    return d
```

3.3 VLIW Packetization

3.3.1 Scoreboarding

The machine contains several constraints that must be resolved in instruction scheduling. This includes data hazards where you read and write side by side on the same register that hasn't been settled, structural limitations where you can only utilize a functioning unit with one instruction at the same time, and memory ordering where you access the memory location before an earlier

modification to the same location has been committed. Furthermore, instruction latencies also introduce timing constraints, as a value produced by an instruction may not be available until several clock cycles later. Given these hardware constraints, a software scoreboard has been implemented to model the true dependencies and hardware limits of the machine. A software scoreboard mimics the behaviour of a hardware scoreboard by tracking register availability, functional unit usage, and memory access timing so the scheduler can determine when each instruction can be safely issued. For every instruction, the scoreboard will record when each register was last written, when each functional unit is available, and when memory locations can be safely accessed. During each instruction check, the scoreboard looks up these records to compute the earliest cycle each instruction can be issued. Assuming an instruction needs a register that will only be ready two cycles later, the scoreboard will delay that instruction's ready time accordingly. Similarly, if an instruction requests the memory unit, the scoreboard accumulates the predicted memory latency to that instruction's timing to ensure that no other memory operation overlaps with it. By adding up a table of predicted instruction latency, the scoreboard can produce a precise ready-time table that maps each instruction to its earliest safe time to execute without violating any data hazards, structural hazards, or memory ordering.

Code Snippet 9: Dependency Tracking Pseudocode (Software Scoreboarding)

```
For each instruction i:
    start = 0
    For each source reg s:
        start = max(start, last_write[s])
    If memory op:
        start = max(start, last_mem_cycle + 1)

    If load/store same addr:    start = max(start, last_store_at[mem_key])
```

```
ready_time[i] = start
last_write[dst] = start + latency(op)
If store: last_store_at[mem_key] = start + latency(op)
If mem op: last_mem_cycle = start
Return ready_time
```

3.3.2 Packetization

The packetizer integrates the software scoreboard into the instruction selection algorithm and groups up to four instructions into VLIW packets that can be executed in parallel. Initially, the packetizer scans through the assembly instruction list cycle by cycle and greedily selects instructions with ready times less than or equal to the current cycle, as shown in Code Snippet 10. After an instruction is selected, a series of hazard checks will be performed. The instruction cannot use a register that another instruction within the same packet writes to. It also cannot request a functional unit that is already being used by another instruction in the same packet. Only one instruction can access the memory unit at the same time to avoid structural hazards. Instructions will be processed on a per-basic-block basis. Basic blocks are separated by control flow instructions like jumps and branches. If no additional instructions can be legally added to the current packet, the rest of the packet will be filled with NOPs, and the packetizer advances to the next cycle. This iterative process continues throughout the entire assembly instruction list and transforms the linear instruction stream into a sequence of tightly packed VLIW packets that can be safely and efficiently executed on the hardware.

Code Snippet 10: Greedy Packing Pseudocode

```

While there are unscheduled instructions:
    Create empty packet for this cycle
    For each instruction in order:
        Skip if already scheduled or not ready yet
        Skip if control mixes with others
        Skip if memory op and one already in packet
        Skip if RAW/WAR/WAW hazard inside packet
        Add instruction to packet
        Mark as scheduled
        If packet has 4, stop
    If packet empty:
        Record empty packet (bubble → 4 NOPs later)
        Advance cycle
    Else:
        Record filled packet
        Advance cycle
Return list of packets

```

Code Snippet 11: Packetization Wrapper Pseudocode

```

Parse assembly → (line, op info) list
Compute ready_time = build_dependency_graph(...)
Compute packet indices = greedy_pack(...)
Convert indices → text packets
Pad each packet to 4 with "nop"
Return packets

```

4. Results

4.1 Scalar Outputs

We compiled a small test file in C, consisting entirely of operations that would be lowered to scalar instructions, and successfully generated assembly for this code. The C program was specifically designed to contain code that would test all scalar instruction types by invoking register, immediate, memory, and branch operations. In the source code file snippet below, add_s, bge_s, addi_s, sw_s, and li_s all show that each of the instruction types is covered and working as intended.

Code Snippet 12: Test C File and Output for Scalar Operations

C Program	Output Assembly	
<pre> int instruct_tests(int a, int b) { int r_acc = 0; int i_adj = a + 4; int i_mask = b & 0xFF; r_acc ^= i_adj ^ i_mask; int mem_buf[2]; mem_buf[0] = r_acc; mem_buf[1] = mem_buf[0] + 1; int mem_val = mem_buf[1]; if (mem_val >= i_adj) { mem_val -= 2; } else if (mem_val == i_adj) { mem_val += 3; } return r_acc + mem_val; } </pre>	<pre> \$ VUseDef \$ addi_s x9, x12, 0 \$ addi_s x9, x13, 0 \$ instruct_tests_block0: \$ jal x0, instruct_tests_block1 \$ instruct_tests_block1: \$ sw_s x9, 36(x8) \$ sw_s x9, 32(x8) \$ li_s x9, 0 \$ sw_s x9, 28(x8) \$ lw_s x9, 36(x8) \$ addi_s x9, x9, 0 \$ addi_s x9, x9, 4 \$ sw_s x9, 24(x8) \$ lw_s x9, 32(x8) \$ addi_s x9, x9, 0 \$ andi_s x9, x9, 255 \$ sw_s x9, 20(x8) \$ lw_s x9, 24(x8) \$ addi_s x12, x9, 0 \$ lw_s x9, 20(x8) \$ addi_s x11, x9, 0 \$ lw_s x9, 28(x8) \$ addi_s x10, x9, 0 \$ xor_s x9, x12, x11 \$ xor_s x9, x10, x9 \$ sw_s x9, 28(x8) \$ lw_s x9, 28(x8) \$ addi_s x9, x9, 0 \$ addi_s x11, x8, 12 \$ li_s x10, 0 \$ li_s x9, 4 \$ mul_s x9, x10, x9 \$ add_s x9, x11, x9 \$ sw_s x9, 0(x9) \$ addi_s x11, x8, 12 \$ li_s x10, 0 \$ li_s x9, 4 \$ mul_s x9, x10, x9 \$ add_s x9, x11, x9 \$ lw_s x9, 0(x9) \$ addi_s x12, x9, 0 \$ addi_s x11, x8, 12 \$ li_s x10, 1 \$ li_s x9, 4 \$ mul_s x9, x10, x9 \$ add_s x10, x11, x9 \$ addi_s x9, x12, 1 \$ sw_s x9, 0(x10) \$ addi_s x11, x8, 12 \$ li_s x10, 1 \$ li_s x9, 4 \$ mul_s x9, x10, x9 \$ add_s x9, x11, x9 \$ lw_s x9, 0(x9) \$ addi_s x9, x9, 0 \$ sw_s x9, 8(x8) </pre>	<pre> instruct_tests_block3 \$ jal x0, instruct_tests_block4 \$ instruct_tests_block2: \$ lw_s x9, 28(x8) \$ addi_s x10, x9, 0 \$ lw_s x9, 8(x8) \$ addi_s x9, x9, 0 \$ add_s x9, x10, x9 \$ addi_s x9, x9, 0 \$ jal x0, instruct_tests_epilog \$ instruct_tests_block3: \$ lw_s x9, 8(x8) \$ addi_s x10, x9, 0 \$ li_s x9, 2 \$ sub_s x9, x10, x9 \$ sw_s x9, 8(x8) \$ jal x0, instruct_tests_block2 \$ instruct_tests_block4: \$ lw_s x9, 8(x8) \$ addi_s x10, x9, 0 \$ lw_s x9, 24(x8) \$ addi_s x9, x9, 0 \$ beq_s x10, x9, instruct_tests_block6 \$ jal x0, instruct_tests_block5 \$ instruct_tests_block5: \$ jal x0, instruct_tests_block2 \$ instruct_tests_block6: \$ lw_s x9, 8(x8) \$ addi_s x9, x9, 0 \$ addi_s x9, x9, 3 \$ sw_s x9, 8(x8) \$ jal x0, instruct_tests_block5 \$ instruct_tests_epilog: \$ addi_s x10, x9, 0 \$ VUseDef </pre>

	<pre> \$ lw_s x9, 8(x8) \$ addi_s x10, x9, 0 \$ lw_s x9, 24(x8) \$ addi_s x9, x9, 0 \$ bge_s x10, x9, </pre>	
--	--	--

4.2 Packetization Example Cases

Code snippet 13 is provided to demonstrate the functionalities of our VLIW packetizer. The two test cases illustrate how assembly instructions are grouped into packets of four while maintaining the data and control flow constraints. In the first example, a series of arithmetic and memory operations is packed so that independent instructions are placed into the same packets, while dependent instructions are separated into different packets bundled as tightly as possible. As an example, the `lw` instruction in line 5 and the `sub` instruction in line 6 both read and write to register `x4`, so they are placed at least three packets away due to the load word instruction's 4-cycle latency. Any packets that cannot be filled with any other instructions are padded with NOPs. In the second example, the packetizer identifies basic blocks by detecting control flow instructions, then performs packetization within each basic block independently. This demonstrates that our packetizer not only schedules according to dependency, but also correctly detects control flow boundaries when constructing the VLIW packets.

latency = {"addi": 1, "add": 1, "sub": 1, "mul": 3, "or": 1, "lw": 3, "sw": 1}

Code Snippet 13: Packetizer Example Cases	
Assembly Code Basic Blocks	VLIW Packets
<pre> addi x1, x0, 5 addi x2, x1, 3 mul x3, x1, x2 sw x1, 0(x2) lw x4, 0(x2) sub x2, x4, x5 sub x4, x3, x2 add x5, x4, x1 </pre>	<pre> Packet 0: addi x1, x0, 5 add x11, x12, x13 add x14, x15, x16 nop Packet 1: addi x2, x1, 3 add x12, x13, x14 </pre>


```

or    x6, x1, x2
L1:
add   x11, x12, x13
add   x12, x13, x14
add   x14, x15, x16

```

```

nop
nop
Packet 2:
mul   x3, x1, x2
sw    x1, 0(x2)
nop
nop
Packet 3:
lw    x4, 0(x2)
nop
nop
nop
Packet 4:
sub   x2, x4, x5
nop
nop
nop
Packet 5:
sub   x4, x3, x2
or    x6, x1, x2
nop
nop
Packet 6:
add   x5, x4, x1
nop
nop
nop

```

```

start:
addi  x1, x0, 1
addi  x2, x0, 2
beq   x1, x2, L_taken
addi  x3, x0, 3
addi  x4, x0, 4
j     L_end

L_taken:
addi  x5, x0, 5
addi  x6, x0, 6

L_end:
addi  x7, x0, 7

```

```

Basic block 0
  Packet 0:
    addi x1, x0, 1
    addi x2, x0, 2
    nop
    nop
  Packet 1:
    beq  x1, x2, L_taken
    nop
    nop
    nop
Basic block 1
  Packet 0:
    addi x3, x0, 3
    addi x4, x0, 4
    nop
    nop
  Packet 1:
    j    L_end
    nop
    nop
    nop
Basic block 2
  Packet 0:
    addi x5, x0, 5
    addi x6, x0, 6
    nop
    nop
Basic block 3
  Packet 0:
    addi x7, x0, 7
    nop
    nop
    nop

```

5: Discussion of Results

As we mentioned previously, PPCI is not the most efficient compiler system in terms of runtime due to its nature as a Python code base. Thus, the runtime performance will be reduced compared to other systems like LLVM. Nevertheless, the runtime does not exceed reasonable limits, usually compiling a 100-line program within less than a second.

We are currently unable to test the correctness and performance of the packetized code on the target hardware, as no simulator has been created for Atalla at this time. We have extensively manually verified the correctness and efficiency of the packetization algorithm, and we expect to have achieved high performance. As soon as a simulator becomes available, we intend to verify our implementation and correct any errors that may arise.

6. Limitations

At the moment of writing this report, the compiler is missing some remaining ISA features, such as outputting vector instructions. In the coming semester, we intend to finish the implementation of the vector portion of the ISA, including ISA patterns for vector instructions. We will also implement SDMA and GEMM for a complete and functioning system. Another point of improvement for us is to implement register banking during the register allocation phase. This is a hardware requirement that we did not have time to address, and is also a significant point of research that we need to complete.

When the compiler is fully implemented and rigorously tested using a simulator, we will be able to consider more compiler optimizations for maximum code efficiency. Our focus next semester will lie mostly in finalizing the compiler correctness, implementing register banking, and adding optimizations. After this, the software team will be able to transition to creating programs for

the processor, including kernels for complex AI calculations, which the compiler should effortlessly and efficiently transform into machine code.

7. Conclusion

In conclusion, this compiler team has made meaningful progress through our research, exploration, design, and results for the Atalla compiler. Our goal was to create a compiler to translate from C to the Atalla-specific machine code, efficiently scheduled into VLIW packets. We first started with extensive compiler research, which led us to choose PPCI as the basis of the compiler. After learning the intricacies of PPCI, we configured full scalar operation support in the frontend, as well as support for our Atalla architecture in the backend by creating instruction tokens, specifying registers, defining instructions, and top-level architecture files for scalar operations. We then completed the implementation of bfloat16 and vector datatypes, as well as GEMM intrinsic function implementation in the frontend, also making significant progress towards full backend vector functionality. Overall, we have a fully functioning compiler for scalar operations in compliance with the Atalla ISA, which is also able to emit these instructions in efficient and correct VLIW packets. We are aiming to complete vector instruction ISA patterns and register banking requirements, as well as any additional optimizations, over the course of the next semester.

Citations

- [1] C. Lattner and V. Adve, “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation,” Jan. 2004. <https://llvm.org/pubs/2004-01-30-CGO-LLVM.pdf>
- [2] GeeksforGeeks, “Abstract Syntax Tree (AST) in Java,” GeeksforGeeks, Aug. 11, 2021. <https://www.geeksforgeeks.org/java/abstract-syntax-tree-ast-in-java/>
- [3] “IR,” *cs.lmu.edu*. <https://cs.lmu.edu/~ray/notes/ir/>
- [4] “Improve your model’s performance with bfloat16,” *Google Cloud Documentation*, 2025. <https://docs.cloud.google.com/tpu/docs/bfloat16>
- [5] “Introduction — ppci 0.5.9 documentation,” *Readthedocs.io*, 2020. <https://ppci.readthedocs.io/en/latest/>.